

GO INTERVIEW & TEST MASTERY INVENTORY

FOUNDATIONAL STRING, ARGUMENT, & PROBLEM-SOLVING ROADMAP

1. COMMAND-LINE ARGUMENTS & PROGRAM INGRESS

FOUNDATIONAL MECHANICS

- Reading raw parameters via `os.Args` slice arrays
- Tracking argument count using the `len(os.Args)` length descriptor
- Bypassing the standard program path execution marker (`os.Args[0]`)
- Parsing flag options using `flag.String`, `flag.Int`, and `flag.Parse`

REQUIRED TARGET PROGRAMS TO WRITE

- **Program PrintArgs:** Prints all incoming argument strings line by line
- **Program CountArgs:** Outputs the exact count of arguments passed, ignoring the binary name
- **Program SumArgs:** Ingests argument strings, converts them to integers, and prints their cumulative sum
- **Program ReverseArgs:** Outputs the provided execution arguments in reverse chronological order

2. STRING ANALYSIS & MUTATION MECHANICS

FOUNDATIONAL MECHANICS

- The read-only byte slice composition behind Go's string header allocation
- The architectural boundaries between ASCII bytes (`byte/uint8`) and Unicode runes (`rune/int32`)
- Iterating strings via indexed loop counters vs. decoding code points with `for i, r := range str`
- Casting mechanics between strings, byte arrays (`[]byte`), and rune slices (`[]rune`)

REQUIRED FUNCTIONS TO IMPLEMENT

- **Function StrLen:** Computes true character count without invoking native macros or library headers
- **Function StrRev:** Reverses character positions right-to-left without corrupting multi-byte Unicode strings
- **Function IsPalindrome:** Checks if an isolated character sequence reads identically forward and backward
- **Function Concat:** Merges multiple independent string buffers into a contiguous memory allocation

3. TYPE TRANSFORMATIONS & ASCII MATH

FOUNDATIONAL MECHANICS

- Mapping numerical glyph characters down to their exact physical ASCII table byte values
- Transforming raw text numeric markers into values via the formula: `digit = int(char - '0')`
- Detecting character limits and invalid character types via ASCII boundary checks (`'0' <= char && char <= '9'`)
- Handling sign variations (detecting positive `'+'` vs. negative `'-'` prefixes in strings)

REQUIRED FUNCTIONS TO IMPLEMENT

- **Function Atoi:** Parses a numeric text string parameter and returns its true integer value
- **Function Itoa:** Converts a physical integer variable into its corresponding string glyph format
- **Function Lowercase:** Converts uppercase ASCII characters to lowercase via byte values (`char + 32`)
- **Function Uppercase:** Converts lowercase ASCII characters to uppercase via byte values (`char - 32`)

4. CORE ALGORITHMIC FUNCTIONS & TEST CLASSICS

FOUNDATIONAL MECHANICS

- Managing pointer addresses and mutative parameters (modifying variable content vs. copying values)
- Tracking tracking index boundaries inside conditional sorting blocks
- Using recursion patterns (functions invoking themselves via temporary stack allocations)

REQUIRED FUNCTIONS & PROGRAMS TO IMPLEMENT

- **Function Swap:** Exchanges the underlying values of two memory cell pointers (*int)
- **Program PrintComb:** Generates and displays all unique, ascending combinations of numbers ($a < b < c$)
- **Function Fibonacci:** Computes the value at a specific index sequence using loop or recursion styles
- **Function Factorial:** Calculates the cumulative product of an integer down to one
- **Function BubbleSort:** Sorts an unsorted slice array of integers into ascending chronological order